

1. Análise no modelo RAM

Parâmetros

C: número total de chunks do corpus.
P: número de candidatos com score > 0 para uma consulta.
m: número de termos distintos da consulta após normalização.
n: número de termos do chunk analisado.
V: tamanho do vocabulário do índice invertido.
S: soma dos tamanhos das listas de postings recuperadas.
k: número de resultados solicitados.
r: número de repetições experimentais.

Operações elementares

No modelo RAM, consideramos como operações elementares:

- acesso a elemento de vetor ou lista;
- comparação entre chaves ou valores;
- atribuição;
- incremento ou decremento de índice;
- inserção, deslocamento ou cópia de elemento em lista;
- consulta e atualização de estruturas de dados;
- chamada e retorno de função;
- chamada recursiva.

Busca linear

A busca linear adotada no protótipo percorre, sequencialmente, todos os C chunks do corpus. Para cada chunk, se calcula a pontuação de relevância e verifica se o resultado é positivo, nesse caso, o par (chunk, score) é incluído na lista de candidatos.

Essa estratégia segue a ideia de busca sequencial, onde os elementos são examinados um a um até que a condição desejada seja verificada (CORMEN et al., 2001, Cap. 2). No projeto, essa lógica está implementada em *src/paa_context/linear_search.py*.

Como todos os chunks são examinados, o custo depende de C. Considerando também o processamento dos termos da consulta e do chunk, a implementação adota:

$$\Theta(C(m + n))$$

com espaço adicional $O(m + n + P)$

Insertion Sort

O Insertion Sort organiza os elementos mantendo uma parte da sequência já ordenada e inserindo cada novo elemento na posição apropriada. Essa estratégia é apresentada por Cormen et al. (2001, Seção 2.1), que utiliza o algoritmo como exemplo inicial de ordenação e análise de corretude.

No projeto, o algoritmo recebe os P candidatos recuperados e os ordena pela chave definida pela equipe: maior *score* primeiro e, em caso de empate, menor *source_order*.

No melhor caso, quando a entrada já está ordenada:

$$\theta(P)$$

enquanto, no caso médio e no pior caso:

$$\theta(P^2)$$

A implementação da equipe cria uma cópia da lista de entrada antes de ordenar, resultando em espaço adicional $O(P)$.

Merge Sort

O Merge Sort segue o paradigma de divisão e conquista: divide o problema em subproblemas menores, resolve cada metade recursivamente e combina os resultados ordenados. Essa estratégia é apresentada por Cormen et al. (2001, Seção 2.3).

Na implementação da equipe, a lista de candidatos é dividida em duas partes, cada uma é processada recursivamente e, por fim, *_merge* combina as duas listas utilizando a ordem definida por *sort_key*.

O custo é representado pela recorrência:

$$T(P) = 2T(P/2) + \theta(P)$$

em que as duas parcelas $T(P/2)$ correspondem às chamadas recursivas e $\theta(P)$ corresponde ao trabalho realizado na etapa de merge. Assim:

$$T(P) = \theta(P \log P)$$

no melhor, médio e pior caso.

Busca binária

A busca binária explora uma sequência previamente ordenada dividindo sucessivamente o intervalo de busca aproximadamente pela metade. Cormen et al. (2001, Cap. 2, exercício/problema sobre binary search) apresenta essa estratégia como uma forma de reduzir o espaço de busca de maneira logarítmica.

No projeto, a busca binária é utilizada na configuração C2 para localizar os termos da consulta no vocabulário ordenado do índice invertido. A própria implementação estabelece como pré-condição que o vocabulário esteja ordenado.

A cada iteração, o algoritmo compara o termo procurado com o elemento central e elimina aproximadamente metade do intervalo restante. Portanto, o melhor caso é:

$$\Theta(1)$$

e o pior caso:

$$O(\log V)$$

sendo V o tamanho do vocabulário. Como a implementação é iterativa, o espaço adicional é $O(1)$.

2. Melhor, pior e caso médio

Algoritmo	Melhor caso	Caso médio	Pior caso	Espaço
Busca linear	$\Theta(C(m + n))$	$\Theta(C(m + n))$	$\Theta(C(m + n))$	$O(m + n + P)$
Insertion Sort	$\Theta(P)$	$\Theta(P^2)$	$\Theta(P^2)$	$O(P)$
Merge Sort	$\Theta(P \log P)$	$\Theta(P \log P)$	$\Theta(P \log P)$	$O(P)$
Busca binária	$\Theta(1)$	$O(\log V)$	$O(\log V)$	$O(1)$

Na busca linear, melhor, médio e pior caso possuem a mesma ordem assintótica porque todos os C chunks são sempre percorridos. No Insertion Sort, o melhor caso ocorre quando a lista já está ordenada, enquanto o pior caso ocorre quando os elementos precisam ser deslocados repetidamente. O Merge Sort mantém custo $\Theta(P \log P)$ independentemente da ordem inicial dos candidatos. Na busca binária, o melhor caso ocorre quando o termo procurado está na posição central do intervalo inicial; nos demais casos, o espaço de busca é sucessivamente reduzido pela metade.

3. Recorrências

Merge Sort

O Merge Sort divide a lista de P candidatos em duas metades, ordena cada metade recursivamente e depois combina os resultados por meio do procedimento de merge. Essa estrutura segue o paradigma de divisão e conquista apresentado por Cormen et al. (2001, Seção 2.3).

A recorrência é:

$$T(P) = 2T(P/2) + \Theta(P)$$

em que:

$2T(P/2)$ representa as duas chamadas recursivas sobre as duas metades

$\Theta(P)$ representa o custo de combinar as duas metades no procedimento *_merge*.

Pela análise da árvore de recorrência, cada nível realiza trabalho total proporcional a P , e existem aproximadamente $\log_2 P$ níveis. Assim:

$$T(P) = \Theta(P \log P)$$

Busca binária

A busca binária reduz aproximadamente pela metade o intervalo de busca a cada iteração. No projeto, ela é utilizada para localizar termos no vocabulário ordenado do índice invertido.

A recorrência associada é:

$$T(V) = T(V/2) + \Theta(1)$$

em que:

- $T(V/2)$ representa a continuação da busca em metade do intervalo
- $\Theta(1)$ representa o trabalho constante de calcular o meio, comparar o termo e atualizar os limites.

Como o tamanho do problema é reduzido pela metade a cada passo:

$$T(V) = \Theta(\log V)$$

4. Análise das configurações C1, C2 e C3

C1 — Busca linear + Insertion Sort

Na configuração C1, todos os C chunks do corpus são percorridos pela busca linear. Os P candidatos com score positivo são então ordenados com Insertion Sort.

Assim:

$$TC1(C, P) = \Theta(C(m + n)) + T_{Insertion}(P)$$

No melhor caso:

$$TC1 = \Theta(C(m + n) + P)$$

No caso médio e no pior caso:

$$TC1 = \Theta(C(m + n) + P^2)$$

C2 — Índice invertido + busca binária + Merge Sort

Na configuração C2, os termos da consulta são localizados no vocabulário ordenado do índice invertido por meio de busca binária. Para m termos da consulta, essa etapa possui custo $O(m \log V)$.

Em seguida, as listas de postings associadas aos termos encontrados são recuperadas e combinadas. Os candidatos resultantes são ordenados com Merge Sort.

De forma simplificada:

$$TC2 = O(m \log V + S + P \log P)$$

em que S representa o custo associado às listas de postings recuperadas.

C3 — Busca linear + Merge Sort

A configuração C3 utiliza a mesma busca linear de C1, mas substitui o Insertion Sort pelo Merge Sort.

Assim:

$$TC3 = \Theta(C(m + n)) + \Theta(P \log P)$$

portanto:

$$TC3 = \Theta(C(m + n) + P \log P)$$

Configuração	Recuperação	Ordenação	Custo simplificado
C1	Busca linear	Insertion Sort	$\Theta(C(m + n) + P^2)$ no pior caso
C2	Índice invertido + busca binária	Merge Sort	$O(m \log V + S + P \log P)$
C3	Busca linear	Merge Sort	$\Theta(C(m + n) + P \log P)$

5. Validação prática da análise de corretude

Como complemento à análise teórica, foi criado o arquivo `tests/unit/test_sort_key.py`, com o objetivo de validar diretamente a relação de ordenação utilizada no projeto e considerada na prova de corretude.

A função `sort_key`, localizada em `src/paa_context/sort_key.py`, estabelece que candidatos com maior score devem aparecer primeiro e, em caso de empate, o menor `source_order` deve ter prioridade.

Foram adicionados três testes:

- `test_maior_score_vem_primeiro()`: verifica a prioridade do maior score;

- `test_empate_usa_menor_source_order()`: verifica o critério de desempate;
- `test_ordem_transitiva()`: verifica um caso de transitividade da relação de ordenação.

Inicialmente, os três testes específicos foram executados isoladamente e todos foram aprovados. Em seguida, foi executada a suíte completa do projeto, obtendo-se **118 testes aprovados em 0,40 s**.

```
(.venv) galvao@fedora:~/Documentos/GitHub/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto_Rafael_Leal_Helena_Silva_Eduardo$ python3 -m pytest tests/unit/test_sort_key.py -v
===== test session starts =====
platform linux -- Python 3.10.14, pytest-8.3.4, pluggy-1.6.0
rootdir: /home/galvao/Documentos/GitHub/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto_Rafael_Leal_Helena_Silva_Eduardo
configfile: pyproject.toml
collected 3 items

tests/unit/test_sort_key.py ... [100%]

===== 3 passed in 0.03s =====
(.venv) galvao@fedora:~/Documentos/GitHub/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto_Rafael_Leal_Helena_Silva_Eduardo$ python3 -m pytest
118 passed in 0.40s [100%]
(.venv) galvao@fedora:~/Documentos/GitHub/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto_Rafael_Leal_Helena_Silva_Eduardo$
```

Execução dos testes específicos de `sort_key` e da suíte completa do projeto.